Bilkent University
Department of Computer Engineering

Senior Design Project

T2528
UppGrad

Detailed Design Report

Ali Özhavala (22203113),

Begüm Filiz Öz (22203470),

Bertan Turgut (22203226),

Emre Uçar (22203675),

Koray Sevil (22202548)

Supervisor: Selim Aksoy

Course Instructors: İlker Burak Kurt, Mert Bıçakçı

Innovation Expert: Alper Sarıkan

13.03.2026

This report is submitted to the Department of Computer Engineering of Bilkent University in partial fulfilment of the requirements of the Senior Design Project course CS491/2.

Table of Contents

1 Introduction.....4
1.1 Purpose of the System.....4
1.2 Design Goals.....5
1.2.1 Usability.....5
1.2.3 Scalability.....5
1.2.4 Maintainability.....5
1.2.5 Privacy.....5
1.3 Definitions, Acronyms, and Abbreviation.....6
1.4 Overview.....6
2 Current System.....7
3 Proposed System.....8
3.1 Overview.....8
3.2 Subsystem Decomposition.....10
3.2.1 Presentation Layer.....10
3.2.2 Application Layer.....11
3.2.3 Database Layer.....12
3.2.4 Repository Layer.....13
3.2.5 External Service Layer.....13
3.3 Persistent Data Management.....13
3.4 Access Control and Security.....14
4 Subsystem Services.....15
4.1 Document Feedback Service.....15
4.1.1 Intake & Classification Agent.....15
4.1.2 Context Assembly Agent.....15
4.1.3 Document Analysis Agent.....15
4.1.4 Synthesis & Planning Agent.....15
4.1.5 Evaluation Agent.....15
4.1.6 Human Review Coordinator.....16
4.1.7 Rewrite Agent.....16
4.2 Auto-apply Service.....16
4.3 Opportunity Recommendation Service.....18
4.4 Webscraping Service.....18
5 Test Cases.....19
5.1 Functional Test Cases.....19
5.2 Non-Functional Test Cases.....44
6 Other Analysis Elements.....50
6.1 Constraints.....50
6.1.1 Implementation Constraints.....50
6.1.2 Economic Constraints.....50

6.1.3 Privacy, Security, and Ethical Constraints....50
6.1.4 User Experience and Usability Constraints.... 51
6.2 Standards....51
7 Teamwork Details....53
7.1 Contributing and functioning effectively on the team.... 53
7.2 Helping creating a collaborative and inclusive environment.... 54
7.3 Taking lead role and sharing leadership on the team.... 55
8 Glossary....56
9 References....58

1 Introduction

1.1 Purpose of the System

University students and recent graduates face a messy process when they try to plan their careers and search for jobs or internships. Relevant information is scattered across company career pages, LinkedIn, job portals, scholarship sites, social media announcements, and internal university mailing lists. Many students keep half updated spreadsheets, notes, or screenshots, and it is easy to miss opportunities or lose track of application status.

Recruiters and employers, on the other side, work with large applicant pools and basic filters. They often rely on manual screening of resumes or generic applicant tracking systems that do not really understand the student context, such as course projects, research experience, competitions, or side projects.

UppGrad aims to act as a bridge between these sides. It is planned as a web based platform that collects and structures opportunities that are relevant for students and recent graduates, builds rich profiles for candidates, and uses intelligent matching and guidance to help both sides. The system focuses on undergraduate and graduate students in computer science and related fields at first, but the design should allow expansion to other disciplines later.

1.2 Design Goals

1.2.1 **Usability**

New student users should be able to register, complete a minimal profile, and receive their first recommended opportunities within a few minutes. Common actions such as updating profile information, applying to an opportunity, and checking application status should be accessible from the dashboard with only a small number of steps. The interface should remain clear, consistent, and easy to follow on both desktop and mobile browsers.

1.2.2 **Performance**

Under normal operating conditions, the main dashboard should load within a few seconds. Search and filtering operations for opportunities should also return results within a similar time range for typical usage. More expensive computations should be handled through background processing where possible, with cached results presented to users efficiently.

1.2.3 **Scalability**

The system should be able to support growth in the number of users (i.e. thousands of users) without major architectural changes. Stateless services should be deployable across multiple instances so that the platform can scale horizontally as student, recruiter, and university usage increases.

1.2.4 **Maintainability**

The codebase should be organised so that developers can understand, test, and update the system without unnecessary effort. Critical components should be covered by automated tests, and a continuous integration pipeline should run these checks regularly to reduce the risk of unnoticed regressions.

1.2.5 **Privacy**

Sensitive information, such as student contact details or evaluation comments, must not be exposed without a valid purpose and proper permission. Users should be given clear controls over profile visibility, including whether recruiters can view the full profile or only an application specific subset.

1.2.6 **Modularity**

The architecture should be divided into clearly separated services and components, with documented boundaries and interfaces. This should allow different parts of the platform, such as recommendation, profile management, application tracking, and analytics, to evolve with limited impact on one another.

1.3 Definitions, Acronyms, and Abbreviation

AI (Artificial Intelligence): A machine based system that can generate outputs such as predictions, recommendations, content, or decisions that can influence real or virtual environments.

API (Application Programming Interface): A defined interface that lets software components communicate, usually through HTTP requests and responses between client and server.

Authentication: The process of proving a user is who they claim to be, for example by checking a password, then creating a logged in session.

Authorization: The process of deciding what an authenticated user is allowed to access or do, based on permissions and roles.

Backend: The server side of the system that handles business logic, data storage, and API endpoints.

CV (Curriculum Vitae): A document summarizing education, skills, and experience used for applications.

Database: A system used to store and query persistent data such as users, opportunities, applications, and notifications.

ML (Machine Learning): Techniques that let systems improve performance by learning patterns from training data rather than only explicit instructions.

RBAC (Role Based Access Control): An access control model where permissions are assigned to roles, and users gain permissions through the roles they have.

UI (User Interface): The screens, forms, and controls a user interacts with.

1.4 Overview

This report describes the detailed design of our system and explains how the earlier analysis decisions were turned into an implementable software structure. It introduces the main design goals and key terms used throughout the report, then reviews the existing solution space before presenting the proposed architecture, its layered organization, subsystem decomposition, and major service responsibilities. It also summarizes the planned verification approach through functional and non functional integration test cases, and discusses the engineering factors, constraints, and standards that influenced the design choices made by the team. The final parts of the report cover how teamwork was carried out, including member contributions, collaboration practices, and the sharing of leadership across different parts of the project.

2 Current System

From the student perspective

- Students search for jobs, internships, scholarships, and competitions through multiple portals such as LinkedIn, Indeed, company career pages, and informal resources such as WhatsApp and Telegram groups.
- They maintain their own resume in LaTeX, update it occasionally, and customize it manually for each application.
- They keep application status in spreadsheets or notes, (sometimes Notion or Google Sheets) or they simply rely on memory and email inbox search.
- They receive general university wide announcements that are not always tailored to their profile or interests.

From the employer and recruiter perspective

- However, many companies use generic applicant tracking systems that collect resumes and allow basic filtering by keywords, experience, or education.
- For student oriented positions, recruiters still receive large numbers of applications that are very similar and spend time on manual screening.
- Ranking of candidates is often based on quick scans of school name, cGPA, and some buzzwords, rather than a structured view of projects, skills, and potential.

From the university perspective

- Career centres often maintain static web pages with announcements, or send emails about events and postings.
- There is limited feedback about which types of roles students apply to, where they succeed, or how their skills evolve over their studies.

These practices have several problems

- Students can easily miss suitable opportunities because they do not see them at the right time.
- Students have difficulty understanding what they are actually competitive for, and which skills or experiences would push them into the range of more selective positions.
- Recruiters have to handle large applicant pools with simple tools and can miss strong but non standard profiles.
- The process is not very transparent, and communication between student, employer, and university is fragmented.

While some existing platforms like LinkedIn, Glassdoor, Lever and more are helpful for it, we aim to create one extensive and comprehensive platform focused on our target audience, new graduates.

UppGrad is intended to bring structure and data driven support to this situation.

3 Proposed System

3.1 Overview

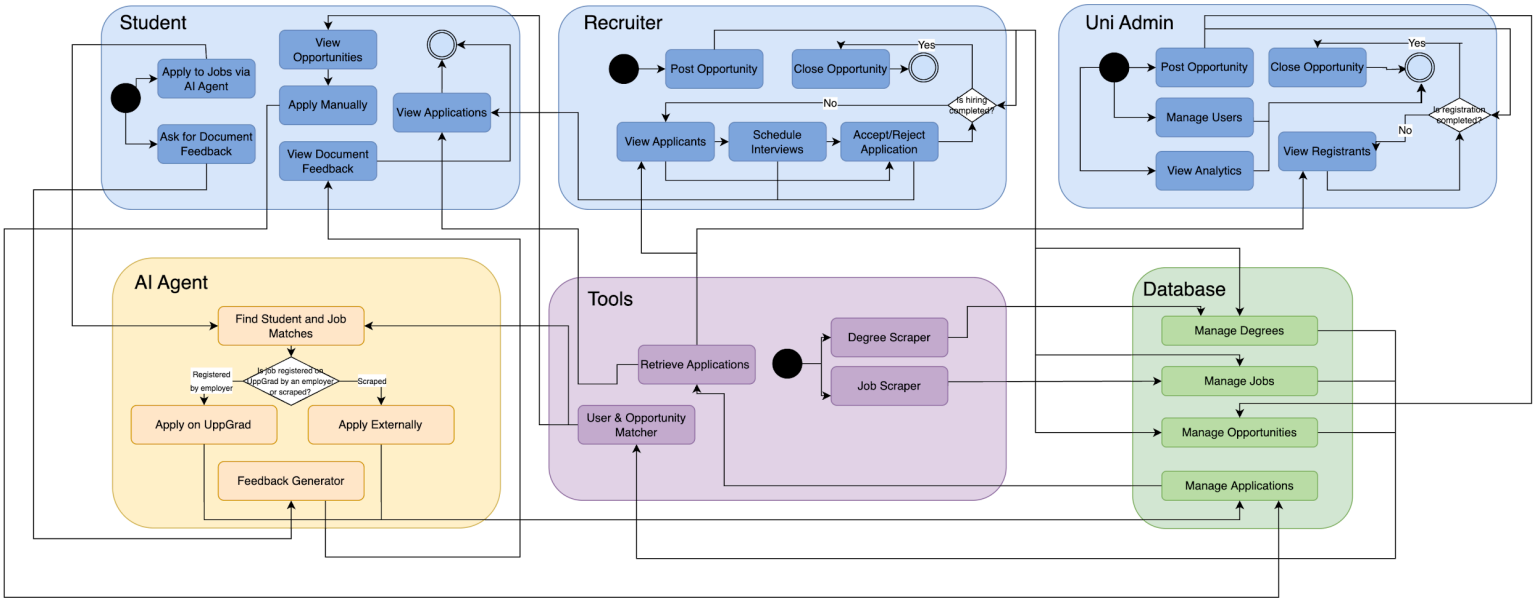

Figure 1: System Overview

As seen in Figure 1, the general overview of UppGrad contains functionalities from the student perspective, and from recruiter and university admin perspectives. To assist the functionalities, the system leverages a variety of tools, an AI agent, and storage using a PostgreSQL database. The main workflows are as follows:

Degree and job scrapers obtain various opportunities from the internet and store them in the database (including jobs, various postgrad degrees, and scholarships). Alternatively, recruiters or university admins can create such postings. In this context, postings can take one of the following forms: they can be a job, a degree, or a general opportunity, which could be a career event, for example. Then, students will apply for these opportunities, jobs, and degrees. Applications can either be done manually through the platform or externally, or through the AI agent, which will be explained in more detail. Manual application can take place on the platform if the job was registered on UppGrad by a recruiter. Otherwise, students would be directed to an external link. For instance, they might apply to degrees via external links. The User & Opportunity matcher tool will show relevant jobs, degrees and opportunities to the student based on their profile.

When students apply to jobs or other opportunities, recruiters and university admin will fetch their applications via the Retrieve Applications tool. This tool will be used to bring up a given student's applications on their "View My Applications" page, and to bring up applicants or registrants on the recruiter/admin's page for the evaluation of the

applicant. All such applications will also be stored in the database and be linked to the corresponding student and opportunity. Recruiters can schedule interviews with applicants and accept/reject them by checking the applicants for their postings. Admins and recruiters can both close their postings when they wish.

As for the university admins, they have some additional features such as Manage Users and View Analytics. Any data from the database can be leveraged for computing analytics for admins. An additional functionality that students have access to is getting document feedback. They can upload their CVs or personal statements on the platform, for which the feedback generator will generate feedback.

Students can also use the AI agent for automated job applications. They can set the frequency and their preferences for agentic applications. The agent will view recommended jobs for the student via the User & Opportunity Matcher as well. If the job is registered on the platform by a recruiter, the agent will apply to the job via UppGrad. Otherwise, and if the source of the external job posting allows it, the agent will apply to the job externally. All applications will still be stored in the database, and the student can view their applications via the “View My Applications” page.

3.2 Subsystem Decomposition

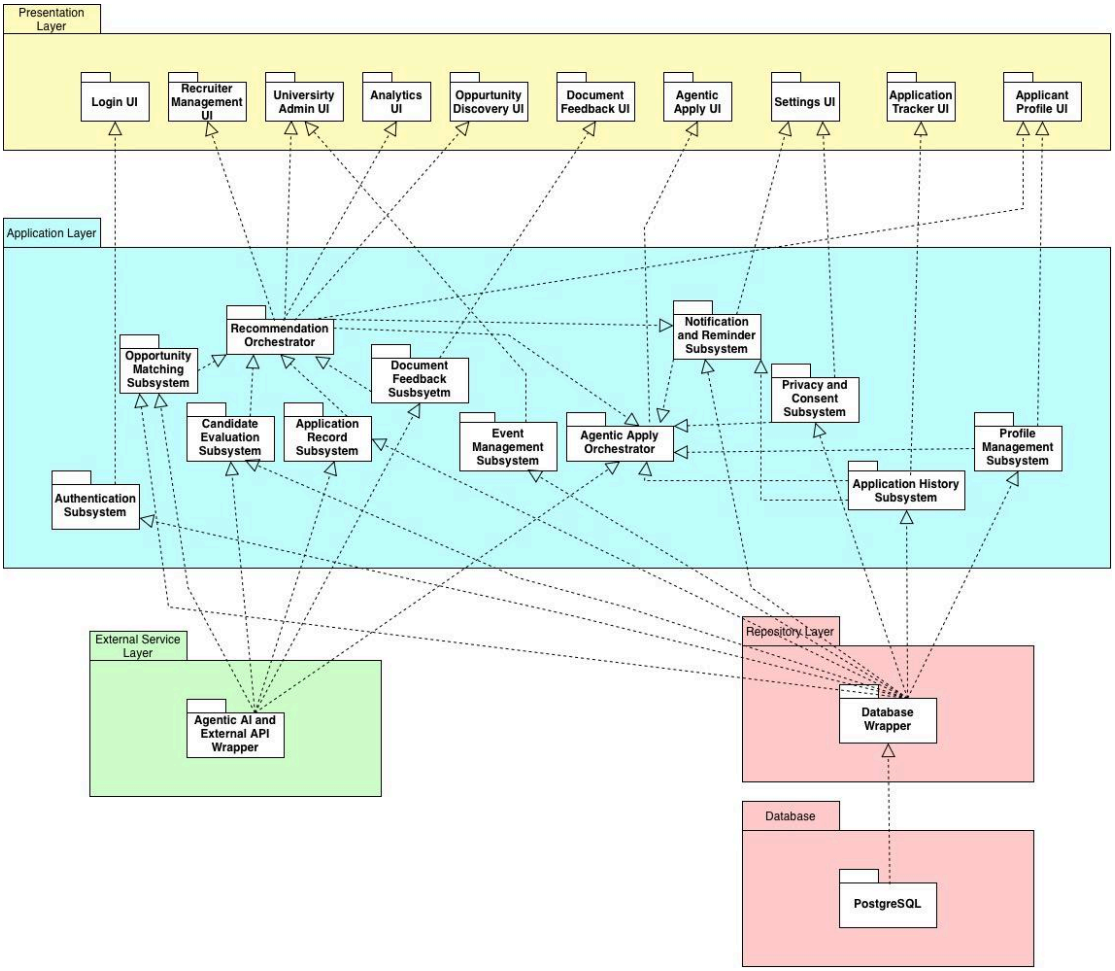

Figure 2: Subsystem Decomposition Diagram

3.2.1 Presentation Layer

The Presentation Layer includes every interface that users directly interact with. Its role is limited to presenting data, collecting user actions, and forwarding those actions to the Application Layer, where the actual system logic is handled.

- **Login UI:** This interface serves as the access gateway to the platform. It provides the forms and entry controls needed for users to sign in before reaching protected system pages.
- **Recruiter Management UI:** This interface is intended for recruiter side operations. Through it, recruiters can publish or update postings, inspect applicants, move candidates through selection stages, and review ongoing hiring activity.
- **University Admin UI:** This interface is intended for university admin side operations. Through it, the professors may inform the admins for a new opening for research to post openings.

- **Analytics UI:** This screen shows summarized statistics and usage indicators for the platform. It allows authorized users to inspect measures such as application activity, recommendation related trends, and general engagement information that support observation of system behavior.
- **Settings UI:** This interface is used to manage adjustable system and account preferences. It allows users to change items such as privacy options, reminder preferences, and other configurable settings that affect their platform experience.
- **Application Tracker UI:** This interface helps students follow the progress of their submitted applications. It presents current statuses, earlier updates, deadline related information, and saved notes or reminders so users can keep track of their application journey in one place.
- **Opportunity Discovery UI:** This interface is used by students to explore available opportunities. It supports browsing, filtering, searching, and opening opportunity details, while also presenting personalized recommendations together with brief explanations of why those opportunities were suggested.
- **Applicant Profile UI:** This interface allows students to manage their personal profile information and uploaded application materials. It supports viewing and updating user details such as skills, interests, academic background, preferences, and resume related content used by other platform services.
- **Document Feedback UI:** This interface allows students to upload resumes and related application documents, view previously submitted files, and access generated feedback. It serves as the user facing entry point for document analysis and improvement suggestions provided by the platform.
- **Agentic Apply UI:** This interface allows students to activate, monitor, and manage the automated application feature of the platform. It presents agent status, activity summaries, and configuration controls related to the automatic application process.

3.2.2 Application Layer

The Application Layer is the main execution layer of the platform. It contains the core services and subsystems that carry out profile handling, recommendation generation, application processing, privacy enforcement, notifications, and access control.

- **Recommendation Orchestrator:** This is the main coordination unit behind personalized opportunity suggestions. It combines profile signals, opportunity information, past user actions, and intermediate matching outputs to generate ranked recommendation results for students.
- **Opportunity Matching Subsystem:** This subsystem performs the comparison between student information and opportunity requirements. It considers items such as role expectations, tags, preferences, and relevant profile attributes, then produces matching outputs that feed into recommendation generation.
- **Candidate Evaluation Subsystem:** This subsystem supports recruiter focused candidate review activities. It processes applicant related data and evaluation criteria

so that recruiters can compare candidates and inspect how well they align with a posting.

- **Application Making Subsystem:** This subsystem manages the main lifecycle of an application after submission. It handles operations such as linking the correct resume, recording application status transitions, and reflecting recruiter side decisions back into the student tracking flow.
- **Profile Management Subsystem:** This subsystem is responsible for handling student profile information. It manages the creation, update, storage, and retrieval of user details such as skills, interests, academic background, preferences, and other profile attributes that are later used by recommendation and matching processes.
- **Notification and Reminder Subsystem:** This subsystem manages messages and reminders generated by the platform. It is responsible for producing alerts related to deadlines, recommendation changes, and application status events, while respecting the notification preferences saved by the user.
- **Privacy and Consent Subsystem:** This subsystem controls how personal data can be exposed within the platform. It applies visibility preferences, consent rules, and access restrictions so that student information is disclosed only in allowed situations.
- **Application History Subsystem:** This subsystem answers requests coming from the Application Tracker UI. It gathers application records, status progress, and related tracking information from storage, then prepares the data in a form suitable for presentation.
- **Authentication Subsystem:** This subsystem handles secure user access. It checks submitted credentials, creates and manages session state, and ensures that protected parts of the platform are available only to authenticated users.
- **Document Feedback Subsystem:** This subsystem is responsible for analyzing uploaded resumes and related application documents. It sends document content for external processing, receives structured feedback, and prepares suggestions related to quality, completeness, skill alignment, and general improvement so that students can strengthen their application materials.
- **Agentic Apply Orchestrator:** This subsystem coordinates the automated application flow of the platform. It uses profile information, ranked opportunities, user preferences, and consent related constraints to identify suitable opportunities, initiate application related actions, and track agent driven activity through the rest of the system.
- **Event Management Subsystem:** This subsystem stores and coordinates the events may be held by the universities' admins. The university admin may enter a new organization and register it to the system and the university students may view.

3.2.3 Database Layer

The Database Layer is responsible for long term persistence. It stores the information required for the platform to continue operating consistently over time.

- **PostgreSQL:** This is the main persistent storage unit of the system. It keeps records such as user accounts, profile details, resumes, opportunities, applications,

recommendation data, notifications, privacy preferences, configuration values, and historical platform activity.

3.2.4 Repository Layer

The Repository Layer separates business services from direct storage access. It provides a cleaner bridge between the logic in the Application Layer and the underlying persistence mechanism.

- **Database Wrapper:** This component provides a unified way for application subsystems to interact with stored data. By centralizing data access behind one interface, it reduces direct dependence on database specific operations and helps keep the application logic cleaner.

3.2.5 External Service Layer

The External Service Layer contains the connectors used to communicate with outside systems. This keeps external integration concerns separate from the platform logic.

- **Agentic AI and External API Wrapper:** This component acts as the unified gateway for external intelligent services and third party integrations used by the platform. It supports tasks such as recommendation related inference, automated application support, document feedback generation, external notification delivery, and communication with outside sources that provide opportunity data or related processing capabilities. External services also include degree and job scrapers.

3.3 Persistent Data Management

The system uses PostgreSQL as the primary relational database to store structured information such as user profiles, job postings, academic programs, and application records. Specifically, Neon PostgreSQL is used with Django unmanaged model wrappings to provide a single, shared source of truth for both the web backend and the independent scraping services. This architecture removes the need for data synchronization pipelines between the scrapers and the application layer.

For semi-structured data, such as job compensation details, contact emails, and scraper metadata, the system utilizes the JSONB data type within PostgreSQL. This allows for flexible storage and efficient querying of nested data without requiring a separate NoSQL database.

Uploaded documents, including student resumes, are managed through a dedicated file storage system. These files are stored as objects in the persistent filesystem and linked to user records in the database.

3.4 Access Control and Security

UppGrad allows account creation for students/recent graduates, employers/recruiters and university admins At the first step of the account creation process, basic credentials are to be submitted, which differ by the created account type. For a student account, the required credentials are name, last name, university mail address and password; for an employer account these are name, last name, company mail address, organization name, job title and a password, and finally for the university admin they are name, last name, university and a password. After the credentials are submitted, the user is directed to the verification page, i.e. the second stage of the account creation process, to complete the registration. Mail authentication is chosen as the verification method for all of the account types: the user can follow the account verification link sent to the mail address. Upon successfully opening the link, the account will be authenticated and the user will be able to use the functionality offered by the UppGrad's system.

4 Subsystem Services

4.1 Document Feedback Service

The Document Feedback Service is responsible for analyzing user-submitted application documents: CVs, Statements of Purpose, and Cover Letters; and producing structured, personalized improvement proposals. The service is orchestrated as a multi-phase LangGraph workflow, combining automated document analysis with a human-in-the-loop approval mechanism to ensure user control over all final changes.

The service is composed of seven agents, each owning a coherent functional responsibility:

4.1.1 Intake & Classification Agent

Serves as the entry point. It accepts the uploaded file, performs text extraction from PDF and DOCX formats, validates that the document meets minimum length and content requirements, classifies the document as a CV, SOP, or Cover Letter, and routes the workflow accordingly. Documents that are irrelevant or malformed are rejected at this stage with an appropriate error response.

4.1.2 Context Assembly Agent

Constructs the working context for all downstream agents. It fetches the user's profile snapshot, parses any free-text instructions provided alongside the upload, and optionally retrieves external opportunity context, such as a job posting or graduate program description, if supplied by the user. Its output is a unified context pack that all analysis agents consume.

4.1.3 Document Analysis Agent

Performs a multi-dimensional evaluation of the document's quality. It runs structure analysis, style and grammar analysis, content gap analysis, and when opportunity context is present, alignment analysis against the target opportunity. For CVs, ATS compatibility is also assessed. These evaluations are executed in parallel and are parameterized by document type, allowing the same agent to apply appropriate rubrics for each format.

4.1.4 Synthesis & Planning Agent

Consolidates the parallel analysis outputs into a coherent diagnostic summary and transforms it into a structured list of change proposals. Each proposal specifies the affected section, a rationale, before and after text, a confidence score, and whether user confirmation is required. This agent is the bridge between critique and actionable editing.

4.1.5 Evaluation Agent

Performs a quality verification pass over the proposed change list before it is presented to the user. It checks for hallucinated additions, ungrounded claims, and format compliance. If proposals fail this check, the agent triggers a revision loop that is capped at two iterations back to the Synthesis & Planning Agent.

4.1.6 Human Review Coordinator

Manages the human-in-the-loop interrupt point. It presents the validated change proposals to the user as a reviewable checklist, collects accept/reject decisions and optional comments, and holds the workflow until explicit approval is received. No edits are applied without passing through this gate.

4.1.7 Rewrite Agent

Applies only the user-approved edits to the original document. It resolves any conflicts between overlapping accepted changes, preserves all rejected segments unchanged, and produces the final rewritten document along with a change diff and summary.

4.2 Auto-apply Service

The Auto-Apply Service automates the preparation and submission of graduate and professional applications on behalf of the user. Given a target opportunity sourced either from the platform's internal database or from an externally scraped posting, the service analyzes the application requirements, assesses the user's eligibility and readiness, generates tailored application materials, and attempts submission. Human oversight is embedded at two explicit points in the workflow to ensure user control over both document selection and final package approval before any submission is attempted.

The service is composed of seven agents:

4.2.1 Opportunity Intelligence Agent

Serves as the analytical entry point for every application. It processes the scraped or stored opportunity content to build a comprehensive understanding of the target, identifying the position type, institution or company, program specifics, and any implicit signals about values, culture fit, or desired candidate profile. In the same pass, it extracts all explicit application requirements, normalizes them into a standard internal schema, and classifies each requirement as either satisfiable by an existing user document, in need of adaptation, requiring generation from scratch, or unsupported by the current system. This dual responsibility of semantic understanding and operational requirement parsing is intentionally unified because both draw on the same source material and their outputs are tightly coupled.

4.2.2 Applicant Eligibility and Readiness Agent

Acts as the workflow's primary gatekeeper. Before any document work begins, it evaluates whether the user is eligible to apply by checking hard constraints such as application deadlines, geographic or visa requirements, and minimum degree qualifications, and also whether the user's profile contains sufficient information to proceed. The agent produces one of four decisions: ready to proceed, ready pending user-supplied missing information, ineligible with an explanation, or flagged for manual review. This gate prevents the workflow from investing in document generation for applications the user cannot or should not submit.

4.2.3 Asset Mapping Agent

Bridges the gap between normalized requirements and the user's existing document library. For each required item, it identifies the most appropriate existing document as a base, determines the depth of tailoring needed ranging from no modification to full regeneration, and flags any requirements for which no suitable asset exists. Where a requirement uses non-standard terminology such as "motivation letter" or "personal statement", the agent resolves it to the closest internal document type and selects the best available source material. The output of this agent is a fully specified work order for the tailoring stage.

4.2.4 Application Tailoring Agent

Is the generative core of the service. It executes the document modification pipeline from the Document Feedback Service, operating in apply-mode rather than feedback-mode, meaning approved changes are applied directly without presenting individual proposals to the user at this stage. For each required document, it produces a tailored version grounded in the user's profile and the opportunity context established earlier. It also generates responses to any structured form fields or short-answer questions identified in the requirements.

4.2.5 Application Evaluation Agent

Performs a quality verification pass over the complete application package before it is presented to the user. It checks each tailored document and generates a form response for groundedness against the user's profile, coverage of the normalized requirements, and absence of hallucinated or unsupported claims. If the package fails this check, a refinement loop is triggered back to the Application Tailoring Agent, capped at a fixed number of iterations.

4.2.6 Human Review Coordinator

Manages the two human-in-the-loop interrupt points in the workflow. The first occurs after the Asset Mapping Agent, where the user is presented with the proposed

document mapping and given the opportunity to override selections, upload additional materials from their local device, or provide supplementary context. The second occurs after the evaluation pass, where the user reviews the complete tailored package including all documents, generated form answers, and detected submission fields, and provides explicit approval before any submission attempt is made. No materials are submitted without passing through this final gate.

4.2.7 Submission Agent

Handles the connection to the external application destination and manages the full range of submission outcomes. Prior to any submission attempt, it assesses feasibility by inspecting the target system for automation barriers such as CAPTCHA, two-factor authentication requirements, or unsupported dynamic form widgets, and classifies the attempt as fully automatable, semi-automated requiring a final user action, or a manual fallback. On a successful automated submission it records the application in the user's history. On failure, it classifies the error, preserves the finalized application package, and provides the user with a structured explanation and all generated materials so that manual submission remains straightforward.

4.3 Opportunity Recommendation Service

The service can be realized through a modular, data driven recommendation pipeline that combines profile signals, behavioral context, opportunity metadata, configurable ranking criteria, and real time scoring updates within a scalable orchestration flow. In practice, the system may use a hybrid matching strategy with feature extraction, preference modeling, signal aggregation, candidate retrieval, relevance scoring, and feedback aware re-ranking, all exposed through a flexible service layer that can evolve as new matching logic is introduced. This approach keeps the implementation extensible, allows configuration based tuning, and supports continuous refinement of recommendation quality without requiring major architectural change.

4.4 Webscraping Service

The scraping component consists of two independent services designed to automatically populate the platform with professional and academic opportunities. The LinkedIn scraper discovers job and internship listings by rotating through industry keywords and global locations, specifically targeting postings from the last 24 hours to ensure data freshness. A "Closed Checker" maintenance sub-service regularly verifies existing entries and marks expired positions as closed in the database.

Complementing this, the MastersPortal scraper handles educational data by extracting detailed Master's and Phd degree program information and scholarships in scheduled batches. It uses a state management system to resume sessions across multiple runs and handles complex parsing for academic details like tuition and duration. Both services share a unified PostgreSQL database with the main backend, allowing newly scraped opportunities to be processed by the recommendation engine and displayed to users immediately without manual synchronization.

5 Test Cases

5.1 Functional Test Cases

5.1.1 Functional TF-01 : Invalid or empty document upload handling

Test ID:	**TF-1**
Category:	Functional
Sub-Category:	File Upload
Objective:	Verify that an unrelated or empty uploaded document is detected and does not proceed as a valid target document for document feedback.
Procedure:	1. Navigate to the document upload page. 2. Upload an empty PDF or a file with no meaningful textual content. 3. Trigger the document feedback workflow. 4. Observe the workflow classification/result.
Expected Outcome:	The system recognizes that the uploaded file does not contain usable document content. The workflow does not continue as a normal CV, cover letter, or SoP analysis. The user is shown a clear validation or failure message indicating that the document could not be analyzed meaningfully.
Severity:	High

Table 1: Test Case TF-1

5.1.2 Functional TF-02 : CV classification and routing

Test ID:	**TF-2**
Category:	Functional
Sub-Category:	File Upload
Objective:	Verify that a CV upload is correctly recognized and routed to the CV feedback path.
Procedure:	1. Navigate to the document upload page. 2. Upload a valid CV in PDF or DOCX format. 3. Trigger the document feedback workflow. 4. Observe the detected document type and returned feedback format.
Expected Outcome:	The system classifies the uploaded file as a CV. The workflow routes the file to the CV-specific analysis path. The returned feedback is relevant to CV structure, strengths, weaknesses, missing sections, or improvement suggestions.
Severity:	High

Table 2: Test Case TF-2

5.1.3 Functional TF-03 : Cover letter classification and routing

Test ID:	**TF-3**
Category:	Functional
Sub-Category:	File Upload
Objective:	Verify that a cover letter upload is correctly recognized and routed to the cover letter feedback path.
Procedure:	1. Navigate to the document upload page. 2. Upload a valid cover letter in PDF or DOCX format. 3. Trigger the document feedback workflow. 4. Observe the detected document type and return feedback.
Expected Outcome:	The system classifies the uploaded file as a cover letter. The workflow routes the file to the correct feedback node. The feedback is tailored to cover letter content rather than CV-style feedback.
Severity:	High

Table 3: Test Case TF-3

5.1.4 Functional TF-04 : Statement of Purpose classification and routing

Test ID:	**TF-4**
Category:	Functional
Sub-Category:	File Upload
Objective:	Verify that a Statement of Purpose upload is correctly recognized and routed to the SoP feedback path.
Procedure:	1. Navigate to the document upload page. 2. Upload a valid Statement of Purpose in PDF or DOCX format. 3. Trigger the document feedback workflow. 4. Observe the detected document type and return feedback.
Expected Outcome:	The system classifies the uploaded file as a Statement of Purpose. The workflow routes the file to the SoP-specific analysis path. The returned feedback is relevant to SoP quality and improvement areas.
Severity:	High

Table 4: Test Case TF-4

5.1.5 Functional TF-05 : Structured document feedback generation

Test ID:	**TF-5**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that structured feedback is returned for a valid uploaded document.
Procedure:	1. Upload a valid CV, cover letter, or SoP. 2. Start the document feedback workflow. 3. Wait for the analysis to complete. 4. Inspect the returned feedback sections.
Expected Outcome:	The system returns structured feedback rather than a vague generic response. The response includes items such as strengths, weaknesses, missing sections, or practical suggestions for improvement. The feedback is display-ready for the interface.
Severity:	High

Table 5: Test Case TF-5

5.1.6 Functional TF-06 : Document to opportunity comparison feedback

Test ID:	**TF-6**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that document-to-opportunity comparison highlights matched and missing points.
Procedure:	1. Upload a valid CV or letter. 2. Open a saved opportunity. 3. Select the uploaded document together with that opportunity. 4. Trigger the tailored feedback workflow.
Expected Outcome:	The system compares the selected document against the opportunity requirements. It highlights points that are well covered. It also highlights weak or missing areas relevant to that opportunity.
Severity:	High

Table 6: Test Case TF-6

5.1.7 Functional TF-07 : Auto apply profile completeness validation

Test ID:	**TF-7**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that auto-apply does not proceed when the user profile lacks required information.
Procedure:	1. Ensure the student profile is missing required contact or core application information. 2. Open an opportunity with auto-apply enabled. 3. Start the auto-apply workflow. 4. Observe validation behavior.
Expected Outcome:	The workflow checks profile completeness before submission. The application is not submitted when required fields are missing. The user receives a clear message indicating that additional profile information is required before continuing.
Severity:	High

Table 7: Test Case TF-7

5.1.8 Functional TF-08 : Resume selection in auto apply

Test ID:	**TF-8**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that auto-apply prompts the student to choose a resume and indicates the primary one.
Procedure:	1. Upload at least two resumes. 2. Mark one as primary. 3. Open an auto-apply-enabled opportunity. 4. Start the auto-apply workflow.
Expected Outcome:	The workflow presents available resumes for selection. The primary resume is clearly indicated as the default or primary option. The user can explicitly choose which resume will be used for submission.
Severity:	High

Table 8: Test Case TF-8

5.1.9 Functional TF-09 : Human confirmation before auto apply submission

Test ID:	**TF-9**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that human-in-the-loop confirmation is required before an auto-apply submission is sent.
Procedure:	1. Open an auto-apply-enabled opportunity. 2. Select a resume. 3. Reach the submission confirmation step. 4. Cancel once, then restart and confirm submission.
Expected Outcome:	The workflow requires explicit user confirmation before sending the application. If the user cancels, no submission is sent. If the user confirms, the workflow proceeds to prepare and attempt the submission.
Severity:	High

Table 9: Test Case TF-9

5.1.10 Functional TF-10 : Successful auto apply record creation

Test ID:	**TF-10**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that a successful auto-apply attempt creates a submitted application record linked to the selected resume.
Procedure:	1. Open an auto-apply-enabled opportunity. 2. Select a specific resume. 3. Confirm submission. 4. After the workflow completes, open the application tracker.
Expected Outcome:	A new internal application record is created. Its status is marked as submitted. The record is linked to the exact resume selected during the workflow. The submission time is stored. A success notification or result message is shown to the user.
Severity:	High

Table 10: Test Case TF-10

5.1.11 Functional TF-11 : Failed auto apply attempt handling

Test ID:	**TF-11**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that a failed auto-apply attempt records the outcome and provides a next-step message.
Procedure:	1. Open an auto-apply-enabled opportunity configured to simulate or return a submission failure. 2. Select a resume. 3. Confirm submission. 4. Observe the result shown to the user.
Expected Outcome:	The workflow records that the submission attempt failed. The user is notified of the failure. The interface provides an appropriate next step such as retry or manual apply. The system does not silently fail.
Severity:	High

Table 11: Test Case TF-11

5.1.12 Functional TF-12 : Non blocking document feedback progress handling

Test ID:	**TF-12**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that document feedback shows progress and does not block general interface usage while processing.
Procedure:	1. Upload a valid long document. 2. Trigger document feedback. 3. While analysis is running, navigate to another page such as dashboard or opportunities. 4. Return to the feedback page after processing completes.
Expected Outcome:	The interface shows that analysis is in progress. The user can continue browsing other parts of the platform while analysis runs. The completed feedback becomes available afterward without losing the request context.
Severity:	Medium

Table 12: Test Case TF-12

5.1.13 Functional TF-13 : Role based username or email label switching

Test ID:	**TF-13**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that the "Username/Email" label and placeholder text update based on the selected role.
Procedure:	1. Navigate to the UppGrad login page. 2. Click on the Student tab. 3. Click on the Employer tab. 4. Click on the Admin tab.
Expected Outcome:	Selecting Student shows the label "Email" and placeholder your.email@university.edu. Selecting Employer shows the label "Username" and placeholder your company username. Selecting Admin shows label "Username" and placeholder your.email@university.edu.
Severity:	High

Table 13: Test Case TF-13

5.1.14 Functional TF-14 : Password visibility toggle behavior

Test ID:	**TF-14**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify the functionality of the "eye" icon in the password field.
Procedure:	1. Enter a value into the Password field 2. Observe the default masking. 3. Click the Eye icon on the right side of the password box.
Expected Outcome:	Clicking the icon should toggle the password visibility between masked and plain text.
Severity:	Medium

Table 14: Test Case TF-14

5.1.15 Functional TF-15 : Remember me checkbox interaction

Test ID:	**TF-15**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that the user can interact with the "Remember me" checkbox.
Procedure:	1. Click on the Remember me checkbox. 2. Refresh the page or navigate away and back.
Expected Outcome:	The checkbox should be selectable/deselectable. If checked, the system should ideally retain the username upon the next visit.
Severity:	Medium

Table 15: Test Case TF-15

5.1.16 Functional TF-16 : Sign up and forgot password redirection

Test ID:	**TF-16**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that the "Sign up" and "Forgot password?" links redirect to the correct pages.
Procedure:	1. Click the Forgot password? link. 2. Go back and click the Sign up link at the bottom.
Expected Outcome:	The user should be redirected to the password recovery page and the account registration page respectively.
Severity:	High

Table 16: Test Case TF-16

5.1.17 Functional TF-17 : Active role visual distinction on login

Test ID:	**TF-17**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that the active role selection is visually distinct from inactive roles.
Procedure:	1. Observe the role selection section. 2. Click on each role one by one.
Expected Outcome:	The active role (e.g., Admin) should have a blue border and blue text, while inactive roles should have a light gray border and dark gray text.
Severity:	Low

Table 17: Test Case TF-17

5.1.18 Functional TF-18 : Empty field validation on sign in

Test ID:	**TF-18**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify system behavior when clicking "Sign In" with empty fields.
Procedure:	1. Leave the Username/Email and Password fields empty. 2. Click the Sign In button.
Expected Outcome:	Error messages should appear indicating that fields are required. The user should not be logged in.
Severity:	High

Table 18: Test Case TF-18

5.1.19 Functional TF-19 : Long and special character input handling in login

Test ID:	**TF-19**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify the input fields handle maximum character lengths and special characters.
Procedure:	1. Select the Student role. 2. Enter a very long string of characters (e.g., 255+) into the Email field. 3. Enter special characters or HTML tags (e.g., <script>) into the Username/Email field. 4. Click Sign In.
Expected Outcome:	The UI should not break or overflow. The system should sanitize input to prevent Script Injection (XSS).
Severity:	High

Table 19: Test Case TF-19

5.1.20 Functional TF-20 : Password visibility consistency during role switching

Test ID:	**TF-20**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that toggling the password visibility remains consistent when switching roles.
Procedure:	1. Select Student and enter a password. 2. Click the Eye icon to show the password in plain text. 3. Switch the role to Employer. 4. Observe the password field.
Expected Outcome:	The password should ideally revert to a masked state for security when the role is changed, or maintain the user's toggle preference consistently.
Severity:	Medium

Table 20: Test Case TF-20

5.1.21 Functional TF-21 : Credential retention during role switching

Test ID:	**TF-21**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify if switching roles clears previously entered credentials.
Procedure:	1. Select Student and type an email and password. 2. Click on Admin. 3. Switch back to Student.
Expected Outcome:	The system should follow the requirement document. Typically, switching roles should not lose user progress unless specified.
Severity:	Low

Table 21: Test Case TF-21

5.1.22 Functional TF-22 : Enter key form submission on login

Test ID:	**TF-22**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify the submitted form when the user presses the "Enter" key on the keyboard.
Procedure:	1. Enter valid credentials in the Username and Password fields. 2. While the cursor is in the Password field, press the Enter key.
Expected Outcome:	The system should trigger the Sign In action exactly as if the button was clicked.
Severity:	Medium

Table 22: Test Case TF-22

5.1.23 Functional TF-23 : Student sign up field and description display

Test ID:	**TF-23**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that the Student role displays the correct unique fields and descriptions.
Procedure:	1. Navigate to the Sign Up page. 2. Select the Student card. 3. Observe the card subtitle and available input fields.
Expected Outcome:	Input fields for First Name, Last Name, Email, Password, and Confirm Password are visible. The placeholder for Email is your.email@university.edu.
Severity:	High

Table 23: Test Case TF-23

5.1.24 Functional TF-24 : Student legal consent and account creation action

Test ID:	**TF-24**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify the final submission button and legal consent for Students.
Procedure:	1. Fill in all mandatory fields as a Student. 2. Click the I agree to the Terms of Service and Privacy Policy checkbox. 3. Observe the action button text.
Expected Outcome:	The primary action button text is "Create Account".
Severity:	High

Table 24: Test Case TF-24

5.1.25 Functional TF-25 : Employer sign up specialized fields and progress bar

Test ID:	**TF-25**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that the Employer role displays specialized corporate fields and a multi-step progress bar.
Procedure:	1. Navigate to the Sign Up page. 2. Select the Employer card. 3. Observe the form layout and progress indicator.
Expected Outcome:	A progress bar at the top shows "1 Basic Info" (Active) and "Verification" (Inactive). Unique fields for Organization (placeholder: Company name) and Job Title (placeholder: e.g., Hiring Manager, HR Director) are displayed. The subtitle "Hiring talented students" appears under the Employer label. The Email placeholder is your.email@company.com.
Severity:	High

Table 25: Test Case TF-25

5.1.26 Functional TF-26 : Employer multi step registration flow

Test ID:	**TF-26**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify the multi-step navigation for Employer registration.
Procedure:	1. Select the Employer role. 2. Complete all "Basic Info" fields. 3. Click the primary action button at the bottom.
Expected Outcome:	The primary action button text is "Continue to Verification". Upon clicking, the user should be advanced to the "Verification" stage of the progress bar.
Severity:	High

Table 26: Test Case TF-26

5.1.27 Functional TF-27 : Password and confirm password mismatch validation

Test ID:	**TF-27**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify the Password and Confirm Password synchronization.
Procedure:	1. Enter a valid password in the Password field. 2. Enter a different value in the Confirm Password field. 3. Attempt to proceed by clicking the primary action button.
Expected Outcome:	The system must show a validation error indicating that passwords do not match.
Severity:	High

Table 27: Test Case TF-27

5.1.28 Functional TF-28 : Unauthenticated direct profile access restriction

Test ID:	**TF-28**
Category:	Functional
Sub-Category:	Profile
Objective:	Verify that unauthenticated users cannot access the Profile page directly.
Procedure:	1. Log out of the system. 2. Enter the Profile page URL directly in the browser. 3. Attempt to load the page.
Expected Outcome:	The user is redirected to the login page with no profile information being shown.
Severity:	High

Table 28: Test Case TF-28

5.1.29 Functional TF-29 : Profile retrieval for authenticated user

Test ID:	**TF-29**
Category:	Functional
Sub-Category:	Profile
Objective:	Verify that profile information is correctly retrieved from the database for a logged-in user.
Procedure:	1. Log in with a valid account. 2. Navigate to the Profile page. 3. Observe the displayed user information.
Expected Outcome:	The system fetches and displays the correct profile information associated with the logged-in account.
Severity:	High

Table 29: Test Case TF-29

5.1.30 Functional TF-30 : Profile update persistence

Test ID:	**TF-30**
Category:	Functional
Sub-Category:	Profile
Objective:	Verify that profile data is correctly stored in the database after editing profile fields.
Procedure:	1. Log in and navigate to the Edit Profile functionality. 2. Modify a profile field. 3. Save the changes. 4. Query the database or refresh the profile page.
Expected Outcome:	The updated profile information is correctly stored and retrieved from the database.
Severity:	High

Table 30: Test Case TF-30

5.1.31 Functional TF-31 : Extremely long profile input handling

Test ID:	**TF-31**
Category:	Functional
Sub-Category:	Profile
Objective:	Verify system behavior when profile fields contain extremely long input values.
Procedure:	1. Open the Edit Profile functionality. 2. Enter extremely long text in fields such as project description or skill entries. 3. Attempt to save the profile.
Expected Outcome:	The system validates input length and prevents system failure, either rejecting the input with an appropriate error message or safely handling the input with default values.
Severity:	Medium

Table 31: Test Case TF-31

5.1.32 Functional TF-32 : Incomplete profile data handling

Test ID:	**TF-32**
Category:	Functional
Sub-Category:	Profile
Objective:	Verify that the system handles incomplete profile data without system errors.
Procedure:	1. Use an account with missing profile fields such as skills or experience. 2. Navigate to the Profile page.
Expected Outcome:	The system loads the profile page successfully and handles missing data gracefully (e.g. by showing appropriate messages about its absence).
Severity:	Medium

Table 32: Test Case TF-32

5.1.33 Functional TF-33 : Valid document upload acceptance

Test ID:	**TF-33**
Category:	Functional
Sub-Category:	Document Upload
Objective:	Verify successful document upload with valid type.
Procedure:	1. Navigate to the Documental Feedback page. 2. Upload a valid document, such as a PDF and DOCX. 3. Submit the upload.
Expected Outcome:	The system accepts the document and adds it to the user's document list.
Severity:	High

Table 33: Test Case TF-33

5.1.34 Functional TF-34 : Valid document upload list update

Test ID:	**TF-34**
Category:	Functional
Sub-Category:	Document Upload
Objective:	Verify successful document upload with valid type.
Procedure:	1. Navigate to the Documental Feedback page. 2. Upload a valid document, such as a PDF and DOCX. 3. Submit the upload.
Expected Outcome:	The system accepts the document and adds it to the user's document list.
Severity:	High

Table 34: Test Case TF-34

5.1.35 Functional TF-35 : User scoped document access isolation

Test ID:	**TF-35**
Category:	Functional
Sub-Category:	Document Upload
Objective:	Verify that uploaded documents are correctly associated with the user account.
Procedure:	1. Upload a document as User A. 2. Log out. 3. Log in as User B. 4. Navigate to the Documental Feedback page.
Expected Outcome:	Uploaded documents are only accessible by User A.
Severity:	High

Table 35: Test Case TF-35

5.1.36 Functional TF-36 : Hidden scraped opportunity before approval

Test ID:	**TF-36**
Category:	Functional
Sub-Category:	Scraping
Objective:	Verify that scraped opportunities remain hidden until reviewed or activated by authorised staff.
Procedure:	1. Log in as University Staff. 2. Scrape an opportunity from an external source. 3. Leave the scraped opportunity unapproved or inactive. 4. Log in as a Student and look for it. 5. Log in as University Staff, activate the opportunity. 6. Log in as a Student, look for the opportunity again.
Expected Outcome:	The student should not be able to see the opportunity until the staff approves and activates it. Afterwards, the student should see the opportunity.
Severity:	Medium

Table 36: Test Case TF-36

5.1.37 Functional TF-37 : Recommendation refresh after job edit

Test ID:	**TF-37**
Category:	Functional
Sub-Category:	Reliability
Objective:	Verify that editing a job posting triggers recommendation recalculation and student notification.
Procedure:	1. Log in as a Recruiter and open an existing job posting. 2. Edit fields such as requirements, skills, or deadline. 3. Save the posting. 4. Log in as an affected Student and check recommendations and notifications.
Expected Outcome:	The system should save the updated posting, recalculate affected recommendation scores, reorder recommendations where needed, and generate a notification for impacted students.
Severity:	Medium

Table 37: Test Case TF-37

5.1.38 Functional TF-38 : Unsupported document format rejection

Test ID:	**TF-38**
Category:	Functional
Sub-Category:	Document Upload
Objective:	Verify system behavior when uploading unsupported file formats.
Procedure:	1. Navigate to the Documental Feedback page. 2. Attempt to upload an unsupported file type, e.g. .mp4.
Expected Outcome:	The system rejects the upload and displays an appropriate error message regarding desired file format.
Severity:	High

Table 38: Test Case TF-38

5.1.39 Functional TF-39 : Oversized document upload rejection

Test ID:	**TF-39**
Category:	Functional
Sub-Category:	Document Upload
Objective:	Verify system behavior when uploading files exceeding the allowed size limit.
Procedure:	1. Navigate to the Documental Feedback page. 2. Attempt to upload a valid file type larger than the allowed size limit.
Expected Outcome:	The system rejects the upload and displays an appropriate error message regarding file size restrictions.
Severity:	High

Table 39: Test Case TF-39

5.1.40 Functional TF-40 : Uploaded document download

Test ID:	**TF-40**
Category:	Functional
Sub-Category:	Document Upload
Objective:	Verify document download functionality for uploaded documents.
Procedure:	1. Navigate to Documental Feedback. 2. Locate an uploaded document. 3. Click the download icon.
Expected Outcome:	The correct document is downloaded successfully without corruption.
Severity:	High

Table 40: Test Case TF-40

5.1.41 Functional TF-41 : Profile based recommendation generation

Test ID:	**TF-41**
Category:	Functional
Sub-Category:	AI Recommendations
Objective:	Verify AI recommendations are generated and displayed based on the student's profile.
Procedure:	1. Log in as a Student with a fully populated profile (skills, education, preferences). 2. Navigate to the Student Home Page / Dashboard. 3. Observe the generated recommendation cards.
Expected Outcome:	Recommendations should be successfully fetched from the recommendation endpoint and should match the student's specific skills and preferences (e.g., showing Software Engineering jobs for a CS student).
Severity:	High

Table 41: Test Case TF-41

5.1.42 Functional TF-42 : Unified mixed opportunity recommendation feed

Test ID:	**TF-42**
Category:	Functional
Sub-Category:	AI Recommendations
Objective:	Verify that the unified recommendation feed includes mixed opportunity types.
Procedure:	1. Log in as a Student. 2. Navigate to the Student Home Page. 3. Scroll through the AI recommendation feed.
Expected Outcome:	The feed must seamlessly display a mix of Job Postings, Master's Programs, Scholarships, and other opportunities, indicating that the unified graph logic works.
Severity:	High

Table 42: Test Case TF-42

5.1.43 Functional TF-43 : Recommendation refresh after profile change

Test ID:	**TF-43**
Category:	Functional
Sub-Category:	AI Recommendations
Objective:	Verify that changes to a student's profile (skills and preferences) trigger a complete refresh of their AI-generated recommendation feed.
Procedure:	1. Log in as a Student and note the current mix of Job Postings and Master's Programs in the AI recommendation feed. 2. Navigate to the Edit Profile section and make changes to the student's core attributes (e.g., completely removing "Software Engineering" skills/interests and replacing them with "Data Science" and "Machine Learning"). 3. Save the profile changes and return to the Home Page recommendation feed
Expected Outcome:	The system successfully detects the profile update, triggers the asynchronous recommendation refresh, and the feed dynamically repopulates with new opportunities.
Severity:	Medium

Table 43: Test Case TF-43

5.1.44 Functional TF-44 : Smart Rank applicant reordering

Test ID:	**TF-44**
Category:	Functional
Sub-Category:	Smart Ranking
Objective:	Verify the "Smart Rank" functionality successfully reorders job applicants.
Procedure:	1. Log in as an Employer and navigate to the Employer Postings Page. 2. Open the Applicants Popup for a job posting with multiple applicants. 3. Click the "Smart Rank" toggle.
Expected Outcome:	The system evaluates applicant resumes against the job description using the AI endpoint and dynamically resorts the applicants from highest match score to lowest.
Severity:	High

Table 44: Test Case TF-44

5.1.45 Functional TF-45 : Employer job creation and search visibility

Test ID:	**TF-45**
Category:	Functional
Sub-Category:	Opportunity Management
Objective:	Verify employers can create jobs that appear in the global search.
Procedure:	1. Log in as an Employer and navigate to the Employer Postings Page. 2. Create a new Job Posting, filling in all required fields, and set it to active. 3. Log in as a Student and search for the exact job title.
Expected Outcome:	The job posting is instantly persisted in the database and is discoverable in the student's search results.
Severity:	High

Table 45: Test Case TF-45

5.1.46 Functional TF-46 : University event creation with capacity limit

Test ID:	**TF-46**
Category:	Functional
Sub-Category:	Opportunity Management
Objective:	Verify university staff can add capacity-limited events.
Procedure:	1. Log in as University Staff and navigate to the Event management section. 2. Create a new Event with a specific capacity limit. 3. Save the program.
Expected Outcome:	The event successfully saves linked to the University, and the capacity metrics (capacity and current_enrollment) are initialized properly.
Severity:	High

Table 46: Test Case TF-46

5.1.47 Functional TF-47 : Posting analytics conversion calculation

Test ID:	**TF-47**
Category:	Functional
Sub-Category:	Opportunity Management
Objective:	Verify the Job Analytics popup accurately calculates application conversion rates.
Procedure:	1. Log in as an Employer and navigate to the Employer Postings Page. 2. Open the Posting Analytics Popup for a job with a known number of views and applicants.
Expected Outcome:	The system correctly aggregates total views and unique applicants, calculating the exact conversion percentage accurately.
Severity:	Medium

Table 47: Test Case TF-47

5.1.48 Functional TF-48 : Deadline reminder notification for bookmarked opportunities

Test ID:	**TF-48**
Category:	Functional
Sub-Category:	Notifications & Alerts
Objective:	Verify the system correctly sends deadline reminders for bookmarked opportunities.
Procedure:	1. Log in as a Student and ensure that "Deadline Reminders" are enabled in the Notification Settings. 2. Navigate to the Opportunities page and bookmark a Job Posting or Scholarship with an application deadline set to 24 hours from the current time. 3. Trigger the send_deadline_reminders background Celery task (or wait for the scheduled job to run). 4. Check the Student's in-app notifications.
Expected Outcome:	The system should generate a notification alerting the student about the impending deadline for their bookmarked opportunity.
Severity:	Medium

Table 48: Test Case TF-48

5.1.49 Functional TF-49 : Resume linkage preservation for submitted applications

Test ID:	**TF-49**
Category:	Functional
Sub-Category:	Application
Objective:	Verify that each submitted application remains linked to the exact resume chosen at submission time.
Procedure:	1. Log in as a Student. 2. Upload at least two resumes and set one as primary. 3. Apply to an opportunity using one selected resume. 4. Upload another resume or change the primary resume. 5. Open the earlier submitted application record.
Expected Outcome:	The submitted application remains linked to the original resume chosen during submission. Later resume uploads or primary resume changes do not alter the resume attached to that earlier application.
Severity:	Low

Table 49: Test Case TF-49

5.1.50 Functional TF-50 : Personal data export generation

Test ID:	**TF-50**
Category:	Functional
Sub-Category:	Profile
Objective:	Verify that a student can export personal platform data successfully
Procedure:	1. Log in as a Student. 2. Navigate to account or privacy settings. 3. Trigger the personal data export action. 4. Wait for the export process to complete and open the result.
Expected Outcome:	The system should generate the student's data export successfully. The exported data should contain only the requesting student's records.
Severity:	Medium

Table 50: Test Case TF-50

5.2 Non-Functional Test Cases

5.2.1 Non functional TN-01 : Login page responsiveness on smaller screens

Test ID:	**TN-1**
Category:	Non-Functional
Sub-Category:	Responsiveness
Objective:	Verify the login card scales correctly on smaller resolutions.
Procedure:	1. Open the login page. 2. Reduce the browser window width to mobile dimensions 3. Observe the "I am a" buttons (Student, Employer, Admin).
Expected Outcome:	The login card should resize without losing content. The role buttons should ideally stack or remain within the container borders.
Severity:	Low

Table 51: Test Case TN-1

5.2.2 Non functional TN-02 : Password field copy restriction

Test ID:	**TN-2**
Category:	Non-Functional
Sub-Category:	Security
Objective:	Verify that the "Copy" function is disabled for the password field.
Procedure:	1. Enter a password. 2. Highlight the masked password and attempt to "Copy" (Ctrl+C). 3. Attempt to paste it into a notepad.
Expected Outcome:	The user should not be able to copy the password from the masked field.
Severity:	Low

Table 52: Test Case TN-2

5.2.3 Non functional TN-03 : Sign in redirection consistency and accessibility

Test ID:	**TN-3**
Category:	Non-Functional
Sub-Category:	Login & Sign In
Objective:	Verify visual consistency and accessibility of the "Sign in" redirection.
Procedure:	1. Scroll to the bottom of either the Student or Employer Sign Up page. 2. Locate the "Already have an account?" text. 3. Click the Sign in link.
Expected Outcome:	The user is immediately redirected to the main Login page.
Severity:	Medium

Table 53: Test Case TN-3

5.2.4 Non functional TN-04 : Posting analytics consistency after stage updates

Test ID:	**TN-4**
Category:	Non-Functional
Sub-Category:	Reliability
Objective:	Verify that posting analytics update correctly after candidate stage changes.
Procedure:	1. Log in as a Recruiter. 2. Open a posting with multiple applicants and view its analytics section. 3. Move one or more candidates between pipeline stages. 4. Refresh the analytics view.
Expected Outcome:	The analytics reflect the updated number of applicants in each stage. Stage conversion values are updated accordingly. The totals remain consistent with the current pipeline data.
Severity:	Medium

Table 54: Test Case TN-4

5.2.5 Non functional TN-05 : Document page performance with many uploaded files

Test ID:	**TN-5**
Category:	Non-Functional
Sub-Category:	Performance
Objective:	Verify system performance when multiple documents exist for a user.
Procedure:	1. Upload or simulate a large number of documents for one user (e.g., 50+). 2. Load the Documental Feedback page.
Expected Outcome:	The document list loads successfully without excessive delays or system failures.
Severity:	Medium

Table 55: Test Case TN-5

5.2.6 Non functional TN-06 : Document status visibility and usability

Test ID:	**TN-6**
Category:	Non-Functional
Sub-Category:	Usability
Objective:	Verify that users can easily identify the status of their uploaded documents.
Procedure:	1. Navigate to Documental Feedback. 2. Upload a document. 3. Observe the document entry in the document list.
Expected Outcome:	The document clearly displays a status indicator such as "Loading Document", "Pending Feedback", or "Reviewed".
Severity:	Medium

Table 56: Test Case TN-6

5.2.7 Non functional TN-07 : Cross browser document upload compatibility

Test ID:	**TN-7**
Category:	Non-Functional
Sub-Category:	Compatibility
Objective:	Verify that document uploads work correctly across different major browsers.
Procedure:	1. Open the application in Chrome, Firefox, and Safari. 2. Navigate to the Documental Feedback page. 3. Upload a document from each browser.
Expected Outcome:	The document upload process works correctly in all tested browsers without errors.
Severity:	Medium

Table 57: Test Case TN-7

5.2.8 Non functional TN-08 : University adoption suitability and marketability

Test ID:	**TN-8**
Category:	Non-Functional
Sub-Category:	Marketability
Objective:	Verify that the platform appears suitable for university adoption through institution specific features.
Procedure:	1. Log in as University Staff. 2. Navigate through university related functions such as restricted postings, events, and aggregated statistics. 3. Observe whether the system communicates a clear reason for a university career centre to use the platform.
Expected Outcome:	The platform should show clear institutional value through university specific functions. University staff should be able to see that the system supports campus specific opportunities, events, and student level insight in an appropriate way. The overall presentation should support adoption by academic organisations.

Severity:	Medium

Table 58: Test Case TN-8

5.2.9 Non functional TN-09 : Feedback service failure recovery and user messaging

Test ID:	**TN-9**
Category:	Non-Functional
Sub-Category:	Integration
Objective:	Verify system behavior when the feedback generation service fails after a document upload.
Procedure:	1. Navigate to the Documental Feedback page. 2. Upload a valid document file. 3. Simulate or trigger a failure in the feedback generation service (e.g., API timeout or service unavailability). 4. Observe the document status in the "Your Documents" section.
Expected Outcome:	The system should not remain indefinitely in a loading state. The document status should update to indicate failure or retry, and the user should receive a clear message that feedback generation could not be completed.
Severity:	High

Table 59: Test Case TN-9

5.2.10 Non functional TN-10 : Extendibility for adding a new opportunity type

Test ID:	**TN-10**
Category:	Non-Functional
Sub-Category:	Extendibility
Objective:	Verify that a new opportunity type can be added without major changes to existing opportunity workflows.
Procedure:	1. Add a new opportunity type to the system, such as Fellowship, using the existing opportunity model and management flow. 2. Create one record of this new type through the admin or recruiter interface. 3. Open the student opportunity listing and detail page. 4. Observe whether the new type appears in the same listing and view structure as current opportunity types.
Expected Outcome:	The system should support the new opportunity type with minimal code changes. Existing job, program, and scholarship flows should continue working correctly. The new type should appear in the shared opportunity workflow without breaking the interface.

Severity:	Medium

Table 60: Test Case TN-10

5.2.11 Non functional TN-11 : Maintainability through diagnostic backend logs

Test ID:	**TN-11**
Category:	Non-Functional
Sub-Category:	Maintainability
Objective:	Verify that backend service logs provide enough information to diagnose a failed workflow.
Procedure:	1. Trigger a controlled failure in a backend workflow, such as recommendation refresh or document feedback generation. 2. Open the backend log output or monitoring view. 3. Inspect the recorded messages for request path, failure point, and error context.
Expected Outcome:	The system should record meaningful logs that help developers identify where the failure occurred. The logged information should be clear enough to support debugging without manually reproducing every internal step.
Severity:	Medium

Table 61: Test Case TN-11

6 Other Analysis Elements

6.1 Constraints

6.1.1 Implementation Constraints

- UppGrad will be delivered as a web application with a React based frontend and a Django based backend.
- The system will integrate an LLM API such as OpenAI or Gemini for AI assisted features, including the chatbot and other automation support.
- Data collection will rely on web scraping and external sources such as job boards and university portals to gather listings.
- Version control and issue tracking will be done with Git and GitHub.
- Persistent storage will use a database such as PostgreSQL or MongoDB, while CV files and similar documents will be stored as uploaded objects.
- Deployment will use cloud hosting such as AWS or Google Cloud, especially if model hosting or heavier workloads become necessary.

6.1.2 Economic Constraints

- Core features should remain free for newly graduating students, and monetization if needed should be delayed and handled carefully, for example ads later, or limited paid add ons.
- Job posting APIs can be expensive, so scraping is preferred early on when allowed and technically stable.
- Free tier cloud resources will be used initially, and scaling decisions will be made only when usage requires it.
- Free datasets can be used for prototyping and optional fine tuning, to reduce training cost.
- Some unavoidable costs exist such as domain registration and possible paid hosting if free tiers are not enough.

6.1.3 Privacy, Security, and Ethical Constraints

- CVs and user profile data must be treated as confidential, and must not be shared with third parties without consent.
- Authentication must follow strong practices, including secure password storage, and multifactor authentication support where feasible.
- The system must protect users from automation mistakes, so any agentic auto apply feature must keep a human in the loop and require user confirmation before submissions.

- Recommendations must avoid algorithmic bias based on protected or irrelevant attributes, and the system should support explanation of results at a high level.
- Chatbot conversations should be handled as sensitive content, stored and accessed carefully, and users should be informed through a clear privacy policy.
- If analytics or model improvement uses user data, opt in and opt out must exist.

6.1.4 User Experience and Usability Constraints

- The platform should stay usable for a student who is busy and stressed, meaning low friction onboarding, clear profile completion guidance, and simple navigation between opportunities, resumes, and applications.
- AI outputs must be presented as assistance, not as unquestionable truth, and the UI should encourage user review and edits, especially for parsed CV fields and generated text.
- Recommendations should be explainable in plain language, so users understand why an opportunity was suggested and what to improve

6.2 Standards

For requirements and documentation quality, we will align our requirements writing and SRS structure with ISO IEC IEEE 29148. This helps us write requirements that are testable, unambiguous, and traceable, instead of mixing design decisions with user needs. It also fits well with our report structure, because it gives a clear separation between functional requirements, quality requirements, constraints, and verification planning.

For software quality, we will use the ISO IEC 25010 quality model as a checklist when we define quality requirements and when we review the system before demo and release. This keeps us honest about things teams often forget, like reliability, performance efficiency, usability, maintainability, and security, rather than focusing only on features.

For security verification, we will use the OWASP Application Security Verification Standard as a baseline for web application controls. This gives us concrete items to test, like authentication controls, access control, input handling, and secure design checks, instead of relying on gut feeling. For password storage and session handling details, we will follow the OWASP Cheat Sheet Series guidance, since those two areas are common failure points and they map directly to our login, signup, and session features.

For accessibility, we will align the user interface with WCAG 2.2 success criteria, at least aiming for the common A and AA expectations where feasible. This matters because our users will interact with dashboards, forms, and document upload flows,

and accessibility issues tend to hide until late unless you check against a standard early.

For modeling and communication, we will keep using UML for class and relationship modeling and for keeping the core domain consistent across frontend and backend. In practice, the standard here is less about formal compliance and more about using UML consistently to reduce confusion during implementation and demo.

Considerations	Effect level	Effect
Public health	**Low**	Indirect effects through guidance, stress, and decision support.
Public safety	**High**	Sensitive documents, account security, and personal data protection.
Public welfare	**Moderate**	Broader access to structured opportunities and tracking tools. Not applicable to every member of society
Global factors	**Low**	We started the project with the aim of expansion globally, however the running for this step will operate locally.
Cultural factors	**Moderate**	Different norms for documents and communication.
Social factors	**Moderate**	Fairness and bias concerns in ranking and guidance.
Environmental factors	**High**	Compute usage and cloud footprint should be controlled. API tokens used for LLMs consume energy more than any code regulation system might cost.
Economic factors	**High**	LLMs cost a lot, making the operational costs for the system increase.

Table 62: Factors that can affect analysis and design

7 Teamwork Details

7.1 Contributing and functioning effectively on the team

The team organized the project according to major system responsibilities and report deliverables. The main areas of work included recommendation flow, opportunity matching, application tracking, recruiter side operations, profile management, document feedback, privacy controls, subsystem decomposition, and test case preparation. Each member was responsible for specific parts of the project, but design decisions that affected multiple subsystems were reviewed together.

A considerable amount of team effort was spent on the document feedback mechanism and the agentic AI related parts of the system. These were not small standalone features. They affected profile handling, external service integration, user facing behavior, storage, subsystem boundaries, and testing. Because of this, the team had to review these parts repeatedly while preparing the architecture, refining the diagrams, and writing the subsystem descriptions. Discussions focused on how uploaded documents should move through the system, how feedback should be returned and stored, how profile data should support both recommendation and feedback generation, and how the external intelligent services should be represented in the design without making the internal flow unclear.

The recommendation flow also required coordinated work across multiple members. The team had to align the Recommendation Orchestrator, Opportunity Matching Subsystem, Profile Management Subsystem, and Notification and Reminder Subsystem so that the written report, the subsystem diagram, and the service logic described the same system. This required repeated revision, especially when new design choices were introduced, such as separating profile management from document feedback or making the external AI related part of the system more explicit in the architecture.

To support this work, the team held regular meetings to review progress, revise naming decisions, discuss feature boundaries, and identify inconsistencies between report sections. Shared folders and common working documents were used for report drafts, architecture diagrams, notes, and references. This made it easier for members to continue each other's work when needed and helped maintain consistency across the report. It also reduced the risk of one section being updated while related sections remained outdated.

The team's workflow was effective because it combined ownership with regular cross checking. Members did not only complete isolated tasks. They also reviewed how their work affected related parts of the project, especially in areas where the feedback mechanism and agentic AI related processing interacted with other subsystems.

7.2 Helping creating a collaborative and inclusive environment

The team maintained a working environment in which technical decisions could be discussed openly and revised when necessary. This was particularly important in parts of the project where the design was less straightforward, especially the document feedback mechanism and the agentic AI related services. These areas involved questions about system responsibility, user trust, usefulness of generated output, and the extent of external service dependence. As a result, they required input from more than one perspective.

Team discussions included not only implementation and architecture concerns, but also whether the produced feedback would actually be useful to students. This was important because a feedback feature can easily become generic or unhelpful if it is treated only as an AI output channel. For that reason, the team reviewed this mechanism from several angles, including user clarity, system flow, data handling, and testability. Members contributed by questioning how feedback should be displayed, what kinds of suggestions should be expected, how the result should connect back to profile improvement, and how the system should avoid presenting low quality output as if it were always reliable.

A similar level of discussion was needed for the agentic AI related parts of the design. The team had to determine what should be handled by the platform itself and what should be delegated to external intelligent services. This affected subsystem placement, the role of the external wrapper, and the overall clarity of the architecture. Open discussion helped prevent vague design choices and made the final structure more understandable to all members.

The team also monitored workload and reassigned work when needed. Some parts of the project demanded more time than initially expected, especially subsystem decomposition, diagram refinement, and the preparation of test cases for AI supported and feedback related flows. In such cases, members supported one another by taking over connected writing tasks, checking report consistency, or helping revise diagrams and definitions. This made the workload more manageable and reduced the chance that one difficult section would delay the rest of the report.

The environment was collaborative because important parts of the project were not kept within one person's area only. Members were expected to review shared architectural decisions, raise concerns when something was unclear, and contribute to sections outside their own primary responsibility when the project required it.

7.3 Taking lead role and sharing leadership on the team

Leadership was shared across the team by giving each member ownership of a specific area of the project. Instead of relying on one fixed leader, the team distributed lead responsibility according to the main parts of the system and the report. This allowed each member to guide an important part of the project while still contributing to shared design decisions.

One member took the lead in documentation and report preparation. This included organizing sections, keeping terminology consistent, revising subsystem descriptions, and making sure that updates in the architecture were reflected throughout the report. Another member led the web development and user interface side, focusing on screen responsibilities, presentation layer structure, and the connection between user actions and backend services.

A separate member led the agentic AI related part of the project. This included decisions about how external intelligent services should be integrated, how recommendation related components should interact with those services, and how the AI related flow should be represented clearly in the system architecture. Another member focused on the document feedback mechanism, including how uploaded resumes and other documents should be processed, how feedback should be returned to users, and how this feature should connect to profile management and recommendation flow.

Leadership was also shared in testing and validation. The member leading this part focused on shaping integration level test cases so that the main flows of the system, such as recommendation updates, profile changes, application tracking, recruiter actions, privacy controls, and document feedback behavior, were covered properly.

Although each member led a different area, these responsibilities were closely connected. Changes in one part often required updates in diagrams, report sections, or related subsystems. For that reason, leadership in the team meant both ownership of a specific area and coordination with the rest of the group.

8 Glossary

AI (Artificial Intelligence): A machine based system that can generate outputs such as predictions, recommendations, content, or decisions that can influence real or virtual environments.

API (Application Programming Interface): A defined interface that lets software components communicate, usually through HTTP requests and responses between client and server.

ASVS (Application Security Verification Standard): A checklist style standard used to verify web application security controls during design, implementation, and testing.

Authentication: The process of proving a user is who they claim to be, for example by checking a password, then creating a logged in session.

Authorization: The process of deciding what an authenticated user is allowed to access or do, based on permissions and roles.

Backend: The server side of the system that handles business logic, data storage, and API endpoints.

CDN (Content Delivery Network): A geographically distributed group of servers that caches content closer to users to improve load time and reliability.

CI/CD (Continuous Integration and Continuous Delivery): Automation practices that merge code changes frequently, run builds and tests, and support releasing software in short iterations.

Container: A standard unit of software that packages code and dependencies so an application runs consistently across environments.

CV (Curriculum Vitae): A document summarizing education, skills, and experience used for applications.

Database: A system used to store and query persistent data such as users, opportunities, applications, and notifications.

ETL (Extract Transform Load): A pipeline that collects data, reshapes or validates it, then stores it in a target system.

GDPR (General Data Protection Regulation): The EU regulation that sets rules for processing personal data, including user rights and security expectations.

JWT (JSON Web Token): A compact, URL safe format for representing claims that can be integrity protected or encrypted.

LLM (Large Language Model): An AI model trained on large amounts of text that can understand and generate language.

MFA (Multi Factor Authentication): An authentication method requiring more than one factor, for example password plus a one time code.

ML (Machine Learning): Techniques that let systems improve performance by learning patterns from training data rather than only explicit instructions.

NoSQL (Not Only SQL): A category of databases that store data in formats other than relational tables, often used for flexible schemas or specific scaling needs.

OAuth 2.0 (OAuth 2.0 Authorization Framework): An authorization framework that lets a third party application obtain limited access to an HTTP service, usually after the user approves access.

OWASP (Open Worldwide Application Security Project): A security community that publishes guidance and standards for building and verifying secure web applications.

PostgreSQL: An open source object relational database system commonly used for reliable transactional storage.

RBAC (Role Based Access Control): An access control model where permissions are assigned to roles, and users gain permissions through the roles they have.

REST (Representational State Transfer): An architectural style where clients operate on resources using representations transferred between components, often implemented over HTTP.

Resume: A specific version of a CV that is uploaded and stored so applications can be linked to what was actually submitted.

SMTP (Simple Mail Transfer Protocol): The Internet standard protocol for transporting email, commonly used for account verification messages.

SQL (Structured Query Language): The standard language used to define data structures and operate on data stored in relational databases.

TLS (Transport Layer Security): A protocol that protects data in transit between client and server against eavesdropping and tampering.

UML (Unified Modeling Language): A standardized visual modeling language used to describe system structure and behavior, for example class diagrams.

UI (User Interface): The screens, forms, and controls a user interacts with.

UX (User Experience): How clear, efficient, and pleasant the product feels while users complete tasks.

WCAG 2.2 (Web Content Accessibility Guidelines 2.2): A W3C recommendation that defines success criteria for making web content more accessible.

9 References

Object-Oriented Software Engineering, Using UML, Patterns, and Java, 2nd Edition, by Bernd Bruegge and Allen H. Dutoit, Prentice-Hall, 2004, ISBN: 0-13-047110-0.

Krashen, S. (2002). The comprehension hypothesis and its rivals. Selected papers from the Eleventh International Symposium on English Teaching/Fourth Pan Asian Conference. pp. 395-404. Taipei: Crane Publishing Company.

Krashen, S. (2004). Applying the comprehension hypothesis: Some suggestions. Paper presented at 13th international symposium and book fair on language teaching (English Teachers Association of the Republic of China), Taipei, Taiwan. Retrieved from http://www.sdkrashen.com/content/articles/eta_paper.pdf.

Rodrigo, V (2006). The amount of input matters: Incidental acquisition of grammar through listening and reading. The International Journal of Foreign Language Teaching, 2 (1).10-13.

Ponniah, R. J. (2008). Free voluntary reading and the acquisition of grammar by adult ESL students. The International Journal of Foreign Language Teaching, 4 (1) 20-22.

ISO, ISO/IEC/IEEE 29148:2018, Systems and software engineering, Life cycle processes, Requirements engineering, Geneva, Switzerland, 2018.

ISO and IEC, ISO/IEC 25010:2011, Systems and software engineering, Systems and software Quality Requirements and Evaluation, System and software quality models, Geneva, Switzerland, 2011.

OWASP Foundation, OWASP Application Security Verification Standard, Version 5.0.0, May 2025.

OWASP Foundation, Password Storage Cheat Sheet, OWASP Cheat Sheet Series.

OWASP Foundation, OWASP Code Review Guide, Version 2, Jul 2017.

R. T. Fielding, Architectural Styles and the Design of Network Based Software Architectures, PhD dissertation, University of California, Irvine, 2000.

W3C, Web Content Accessibility Guidelines (WCAG) 2.2, W3C Recommendation, 5 Oct 2023.

National Institute of Standards and Technology, Artificial Intelligence Risk Management Framework AI RMF 1.0, NIST AI 100 Part 1, Jan. 2023.

Organisation for Economic Co operation and Development, OECD Framework for the Classification of AI Systems, OECD Artificial Intelligence Papers, 2022.

Internet Engineering Task Force, The OAuth 2.0 Authorization Framework, RFC 6749, Oct. 2012.

Internet Engineering Task Force, JSON Web Token JWT, RFC 7519, May 2015.

Internet Engineering Task Force, The Transport Layer Security TLS Protocol Version 1.3, RFC 8446, Aug. 2018.

Internet Engineering Task Force, Simple Mail Transfer Protocol, RFC 5321, Oct. 2008.

Internet Research Task Force, Argon2 Memory Hard Function for Password Hashing and Proof of Work Applications, RFC 9106, Sep. 2021.

Object Management Group, Unified Modeling Language, Version 2.5.1, Dec 2017.

European Union, Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016, Official Journal of the European Union, 2016.

Docker, Inc., What is a container, Docker Documentation.

Cloudflare, What is a content delivery network CDN, Cloudflare Learning Center.

K. Schwaber and J. Sutherland, The Scrum Guide, Nov 2020.

GitHub, Inc., About protected branches, GitHub Docs.

GitHub, Inc., About pull request reviews, GitHub Docs.

Association for Computing Machinery, ACM Code of Ethics and Professional Conduct, 22 Jun 2018.

IEEE, IEEE Code of Ethics, approved Aug 1990.